

CRIPTOGRAFIAS NO PHP

Juan Conceição nº19

2º C¹

Introdução

Criptografia é o conjunto de técnicas usadas para proteger informações, impedindo que pessoas não autorizadas as compreendam. No PHP, os principais recursos para isso são `password_hash()`, `password_verify()`, `hash()`, `hash_hmac()` e as extensões OpenSSL e Sodium.

Criptografia x Hash

Criptografia é reversível. Ela é um texto original que vira um texto cifrado e, com a chave certa, pode voltar ao original. Hash funciona como uma "impressão digital" dos dados, o conteúdo passa por uma função matemática e gera um resultado que, em geral, não pode ser revertido para o texto original. Por isso hash e criptografia não são a mesma coisa.

Hash são mais adequados para usar em senhas. Guardar senhas em texto puro no banco de dados é inseguro, pois qualquer acesso indevido exporia as senhas de todos os usuários. O correto é guardar um hash da senha. No login, o PHP compara a senha digitada com esse hash, sem nunca armazenar a senha original.

`password_hash()` e salt

A função `password_hash()` gera um hash seguro para senhas:

```
$hash = password_hash($senha, PASSWORD_DEFAULT);
```

Cada execução gera um resultado diferente para a mesma senha por causa do salt, um valor aleatório combinado com a senha antes do hash. Isso dificulta ataques com tabelas pré-calculadas. O PHP gerencia o salt automaticamente.

`password_verify()`

Usada para checar a senha no login, comparando-a com o hash salvo:

```
if (password_verify($senhaDigitada, $hashSalvo)) {  
    echo "Senha correta!";  
}
```

```
}
```

Se a senha corresponder ao hash, o resultado é true e o acesso pode ser liberado.

hash(), SHA-256, MD5 e SHA-1

A função hash() gera hashes com diferentes algoritmos, como o SHA-256, que produz um resultado de 64 caracteres hexadecimais e é usado para verificar integridade de dados — mas não deve ser usado sozinho para senhas. MD5 e SHA-1 são algoritmos antigos e rápidos demais, o que os torna vulneráveis a ataques de força bruta, por isso não são adequados para armazenar senhas.

Argon2 e PASSWORD_ARGON2ID

PASSWORD_DEFAULT deixa o PHP escolher o algoritmo mais apropriado. Já o Argon2, acessado por PASSWORD_ARGON2ID, é uma família de funções mais moderna, projetada para exigir mais processamento e memória, dificultando ataques de força bruta.

OpenSSL e Sodium

OpenSSL oferece openssl_encrypt() e openssl_decrypt() para criptografar e descriptografar dados que precisam ser recuperados depois. Sodium é uma extensão mais moderna, com funções como sodium_crypto_secretbox() e sodium_crypto_secretbox_open(), usadas para criptografia autenticada com chave secreta.

Criptografia simétrica e assimétrica

Na criptografia simétrica, a mesma chave criptografa e descriptografa os dados (ex.: AES, ChaCha20) o desafio é proteger essa chave. Na criptografia assimétrica, existe um par de chaves: a pública, que pode ser compartilhada, e a privada, que deve ficar em segredo. É usada em certificados digitais e comunicação segura.

Criptografia autenticada e HMAC

Criptografia autenticada garante confidencialidade (só quem é autorizado entende os dados) e integridade (dá para detectar alterações indevidas). O HMAC (hash_hmac()) combina uma função de hash com uma chave secreta para gerar um código que permite verificar se uma mensagem é íntegra e autêntica.

Confidencialidade, integridade e autenticidade

Confidencialidade é impedir acesso não autorizado; integridade é garantir que os dados não foram alterados indevidamente; autenticidade é poder verificar a origem da informação. Esses três pilares sustentam qualquer sistema seguro.

Banco de dados e HTTPS

Senhas nunca devem ir para o banco em texto puro, apenas o hash gerado por `password_hash()`. Além disso, a comunicação entre navegador e servidor deve ser protegida por HTTPS/TLS, o que é diferente de proteger a senha armazenada: um cuida dos dados em trânsito, o outro dos dados guardados.

Criptografia não resolve tudo

Usar criptografia não torna um sistema totalmente seguro. É preciso também se proteger contra SQL Injection, XSS, CSRF, falhas de controle de acesso, problemas de sessão e cookies, bibliotecas desatualizadas e versões antigas do PHP.

Boas práticas e erros a evitar

Boas práticas: usar `password_hash()` e `password_verify()` para senhas; usar OpenSSL ou Sodium para dados que precisam ser recuperados; usar HTTPS/TLS na comunicação; usar consultas preparadas (PDO) para evitar SQL Injection; manter tudo atualizado e proteger as chaves criptográficas.

Erros a evitar: guardar senha em texto puro; usar `md5()` ou `sha1()` para senhas; criar algoritmos de criptografia próprios; deixar chaves expostas no código-fonte ou em repositórios públicos.

Onde é usada

Esses mecanismos aparecem em bancos e sistemas financeiros, lojas virtuais, sistemas escolares, sistemas de saúde e, de forma geral, em qualquer aplicação web que precise proteger senhas, sessões, tokens e dados de autenticação.

Conclusão

A criptografia é essencial para proteger informações em sistemas PHP. Senhas devem usar `password_hash()/password_verify()` com salt dados que precisam ser recuperados usam OpenSSL ou Sodium; e a comunicação deve ser protegida por HTTPS/TLS. Nenhuma dessas ferramentas funciona sozinha, a segurança de uma aplicação depende de vários mecanismos trabalhando juntos, incluindo boas práticas contra SQL Injection, XSS e CSRF.